Πρώτο Σετ Ασκήσεων
Δομές Δεδομένων 2025-2026

Ανδριόπουλος Ιωάννης ΑΜ: 1115519
Καραπατάκης Ευθύμιος ΑΜ: 1115521
Χριστοπούλου Αγγελική ΑΜ: 1115471

Περιεχόμενα

1 Άσκηση 1: Ελεγκτής Έκφρασης με Στοίβα **3**
1.1 Εκφώνηση . . . 3
1.2 Ανάλυση . . . 3
1.2.1 Σύγκριση Υλοποιήσεων . . . 3
1.3 Αλγόριθμος . . . 3
1.4 Υλοποίηση . . . 3
1.5 Αποτελέσματα Εκτέλεσης . . . 4
1.6 Συμπεράσματα . . . 4

2 Άσκηση 2: Δυαδικό Δέντρο Αναζήτησης (BST) **5**
2.1 Εκφώνηση . . . 5
2.2 Ανάλυση . . . 5
2.3 Σύγκριση Υλοποιήσεων . . . 5
2.3.1 Κομβοπροσανατολισμένο Δέντρο (επιλέχθηκε) . . . 5
2.3.2 Φυλλοπροσανατολισμένο Δέντρο . . . 5
2.4 Βασικές Συναρτήσεις . . . 5
2.4.1 Αναζήτηση . . . 5
2.4.2 Εισαγωγή . . . 6
2.4.3 Διαγραφή . . . 6
2.5 Δεδομένα Εισόδου . . . 7
2.6 Αποτελέσματα Εκτέλεσης . . . 7
2.7 Συμπεράσματα . . . 7

3 Άσκηση 3: Διαπεράσεις Δυαδικού Δέντρου **8**
3.1 Εκφώνηση . . . 8
3.2 Ανάλυση . . . 8
3.3 Διαπέραση κατά Επίπεδα (Level Order / BFS) . . . 8
3.3.1 Αλγόριθμος . . . 8
3.3.2 Υλοποίηση . . . 8
3.4 Προδιάταξη (Preorder) . . . 9
3.5 Μεταδιάταξη (Postorder) . . . 9
3.6 Υλοποίηση Ουράς . . . 9
3.6.1 Γραμμική vs Κυκλική Ουρά . . . 10

3.7 Αποτελέσματα Εκτέλεσης 10
3.8 Συμπεράσματα 10

4 Άσκηση 4: Ύψος και Βάθος Δυαδικού Δέντρου 11
4.1 Εκφώνηση 11
4.2 Ανάλυση 11
4.2.1 Ύψος Δέντρου 11
4.2.2 Βάθος Κόμβου 11
4.2.3 Σχέση Ύψους και Βάθους 11
4.3 Υλοποίηση 11
4.3.1 Ύψος Δέντρου 11
4.3.2 Βάθος Κόμβων 12
4.4 Δεδομένα Εισόδου 12
4.5 Αποτελέσματα Εκτέλεσης 13
4.6 Συμπεράσματα 14

5 Άσκηση 5: Ανάπλαση Δυαδικού Δέντρου από Διαπεράσεις 15
5.1 Εκφώνηση 15
5.2 Θεωρητική Ανάλυση 15
5.2.1 Λογική Ανακατασκευής 15
5.2.2 Παράδειγμα Χειροκίνητης Ανακατασκευής 15
5.3 Αλγόριθμος Ανακατασκευής από Preorder + Inorder 16
5.4 Αλγόριθμος Ανακατασκευής από Postorder + Inorder 16
5.5 Επιβεβαίωση Ορθότητας 17
5.6 Δεδομένα Ελέγχου και Αποτελέσματα 17
5.7 Συμπεράσματα 18

1 Άσκηση 1: Ελεγκτής Έκφρασης με Στοίβα

1.1 Εκφώνηση

Υλοποιήστε συνάρτηση που χρησιμοποιεί στοίβα χαρακτήρων για να ελέγξει αν μια έκφραση περιέχει σωστά τοποθετημένες παρενθέσεις (), αγκύλες [] και άγκιστρα {}. Η συνάρτηση πρέπει να επιστρέφει 1 αν η έκφραση είναι σωστή, διαφορετικά 0.

1.2 Ανάλυση

Για την επίλυση χρησιμοποιείται **στοίβα** (stack), η οποία ακολουθεί τη λογική **LIFO** (Last In, First Out): το τελευταίο στοιχείο που εισήχθη είναι το πρώτο που αφαιρείται.

Η στοίβα είναι η κατάλληλη δομή για αυτό το πρόβλημα γιατί κάθε κλειστή παρένθεση πρέπει να αντιστοιχεί στην πιο πρόσφατα ανοιχτή — ακριβώς η συμπεριφορά που προσφέρει το LIFO.

1.2.1 Σύγκριση Υλοποιήσεων

Η στοίβα μπορεί να υλοποιηθεί με δύο τρόπους:

- **Πίνακας:** Απλή και αποδοτική υλοποίηση με σταθερό μέγεθος. Η πρόσβαση γίνεται μέσω δείκτη `pos` που δείχνει στην κορυφή. Μειονέκτημα: σταθερό μέγιστο μέγεθος.
- **Συνδεδεμένη Λίστα:** Δυναμικό μέγεθος χωρίς περιορισμό. Κάθε κόμβος περιέχει έναν χαρακτήρα και δείκτη στον επόμενο. Μειονέκτημα: επιπλέον μνήμη για δείκτες και `malloc`/`free`.

Στην παρούσα υλοποίηση επιλέχθηκε **πίνακας** για απλότητα και αποδοτικότητα.

1.3 Αλγόριθμος

1. Διατρέχουμε την έκφραση χαρακτήρα-χαρακτήρα
2. Αν ο χαρακτήρας είναι ανοιχτή παρένθεση `(`, `[`, `{` → την εισάγουμε στη στοίβα (push)
3. Αν είναι κλειστή παρένθεση:
   - Αν η στοίβα είναι άδεια → λάθος (επιστροφή 0)
   - Αν το τελευταίο στοιχείο ταιριάζει το αφαιρούμε (pop) αλλιως λάθος (επιστροφή 0)
4. Στο τέλος: αν η στοίβα είναι άδεια → σωστό (1), αλλιώς λάθος (0)

1.4 Υλοποίηση

```c
int check(char* ekfrash) {
        char stack[LENGTH];
        int pos = -1;
        int len = strlen(ekfrash);

        for (int i = 0; i < len; i++) {
                char ch = ekfrash[i];

```

```
                if (ch == '(' || ch == '[' || ch == '{') {
                        stack[++pos] = ch;
                }
                else if (ch == ')' || ch == ']' || ch == '}') {

                if (pos == -1) return 0;

                char prohgoymeno = stack[pos--];

                if ((ch == ')' && prohgoymeno != '(') ||
                (ch == ']' && prohgoymeno != '[') ||
                (ch == '}' && prohgoymeno != '{')) {
                        return 0;
                }
                }
        }
        return (pos == -1) ? 1 : 0;
}
```

1.5 Αποτελέσματα Εκτέλεσης

Σχήμα 1: Εκτέλεση του προγράμματος με λάθος input

Σχήμα 2: Εκτέλεση του προγράμματος με σωστό input

1.6 Συμπεράσματα

Η στοίβα αποτελεί την ιδανική δομή για προβλήματα αντιστοίχισης συμβόλων, καθώς η LIFO συμπεριφορά της ταιριάζει φυσικά με την εμφώλευση παρενθέσεων. Η υλοποίηση με πίνακα προτιμάται για απλότητα όταν το μέγεθος εισόδου είναι γνωστό, ενώ η λίστα προσφέρει ευελιξία για άγνωστο μέγεθος.

2 Άσκηση 2: Δυαδικό Δέντρο Αναζήτησης (BST)

2.1 Εκφώνηση

Υλοποιήστε συνάρτηση με το όνομα `binaryTreeSearch` που επιχειρεί να εντοπίσει μια καθορισμένη τιμή σε ένα δυαδικό δέντρο αναζήτησης με ακέραιους. Η συνάρτηση δέχεται ως ορίσματα τη ρίζα του δέντρου και το κλειδί αναζήτησης. Αν βρεθεί, επιστρέφει δείκτη στον κόμβο, αλλιώς `NULL`.

Υλοποιήστε επίσης συναρτήσεις εισαγωγής και διαγραφής. Να γίνει έλεγχος με κύριο πρόγραμμα. Να εξηγηθεί θεωρητικά η διαφορά κομβο- και φυλλοπροσανατολισμένου δέντρου.

2.2 Ανάλυση

Το **Δυαδικό Δέντρο Αναζήτησης** (BST) είναι δομή δεδομένων όπου για κάθε κόμβο ισχύει:

- Όλοι οι κόμβοι του **αριστερού** υποδέντρου έχουν τιμή **μικρότερη** από τον κόμβο
- Όλοι οι κόμβοι του **δεξιού** υποδέντρου έχουν τιμή **μεγαλύτερη** από τον κόμβο

2.3 Σύγκριση Υλοποιήσεων

2.3.1 Κομβοπροσανατολισμένο Δέντρο (επιλέχθηκε)

Κάθε κόμβος — εσωτερικός και φύλλο — αποθηκεύει δεδομένα. Η αναζήτηση μπορεί να τερματίσει σε οποιονδήποτε κόμβο.

- **Πλεονέκτημα:** Απλούστερη δομή, λιγότεροι κόμβοι
- **Μειονέκτημα:** Πιο πολύπλοκη διαγραφή (3 περιπτώσεις)

2.3.2 Φυλλοπροσανατολισμένο Δέντρο

Τα δεδομένα αποθηκεύονται **μόνο στα φύλλα**. Οι εσωτερικοί κόμβοι περιέχουν μόνο τιμές σύγκρισης για την καθοδήγηση της αναζήτησης.

- **Πλεονέκτημα:** Απλούστερη διαγραφή (αφαίρεση φύλλου)
- **Μειονέκτημα:** Διπλάσιοι περίπου κόμβοι, πιο περίπλοκη εισαγωγή

Αν επιλεγόταν φυλλοπροσανατολισμένο δέντρο, η δομή του κόμβου θα είχε επιπλέον πεδίο για διάκριση εσωτερικού/φύλλου, και η αναζήτηση θα κατέληγε πάντα σε φύλλο.

2.4 Βασικές Συναρτήσεις

2.4.1 Αναζήτηση

Αναδρομική συνάρτηση που εκμεταλλεύεται την ιδιότητα BST:

```
struct Node* binaryTreeSearch(struct Node* node, int target) {
        if (node == NULL)       return NULL;
        if (node->data == target) return node;

        if (target < node->data)
```

```
        return binaryTreeSearch(node->left, target);
        else
        return binaryTreeSearch(node->right, target);
}
```

2.4.2 Εισαγωγή

Βρίσκει αναδρομικά την κατάλληλη κενή θέση και τοποθετεί νέο κόμβο:

```
struct Node* insert(struct Node* node, int value) {
        if (node == NULL) return createNode(value);

        if (value < node->data)
        node->left = insert(node->left, value);
        else if (value > node->data)
        node->right = insert(node->right, value);
        else
        printf("The value %d is already in the tree!\n", value);

        return node;
}
```

2.4.3 Διαγραφή

Η διαγραφή αντιμετωπίζει τρεις περιπτώσεις:

1. **Φύλλο (0 παιδιά):** Απελευθερώνεται η μνήμη, επιστρέφεται `NULL`
2. **Ένα παιδί:** Ο γονέας συνδέεται απευθείας με το παιδί
3. **Δύο παιδιά:** Αντικαθίσταται η τιμή με τον *inorder successor* (ελάχιστο του δεξιού υποδέντρου) και διαγράφεται ο successor

```
struct Node* deleteNode(struct Node* node, int value) {
        if (node == NULL) return NULL;

        if (value < node->data)
        node->left = deleteNode(node->left, value);
        else if (value > node->data)
        node->right = deleteNode(node->right, value);
        else {
                if (node->left == NULL && node->right == NULL) {
                        free(node); return NULL;
                }
                else if (node->left == NULL) {
                        struct Node* temp = node->right;
                        free(node); return temp;
                }
                else if (node->right == NULL) {
                        struct Node* temp = node->left;
                        free(node); return temp;
```

```
                }
                else {
                        struct Node* successor = findMin(node->right);
                        node->data = successor->data;
                        node->right = deleteNode(node->right, successor->data);
                }
        }
        return node;
}
```

2.5 Δεδομένα Εισόδου

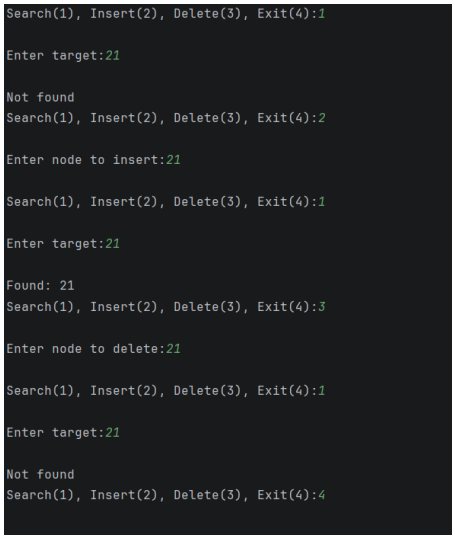
Το δέντρο κατασκευάζεται από το αρχείο `numbers2.txt`:

$$10, 20, 30, 40, 50, 60$$

Οι τιμές εισάγονται σειριακά. Επειδή είναι ήδη ταξινομημένες, το δέντρο εκφυλίζεται σε **λίστα** (skewed tree) με ύψος 5, που αποτελεί τη **χειρότερη περίπτωση** για BST:

$$10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60$$

2.6 Αποτελέσματα Εκτέλεσης



Σχήμα 3: Εκτέλεση λειτουργιών στο δυαδικό δέντρο

2.7 Συμπεράσματα

Το BST προσφέρει αποδοτική διαχείριση δεδομένων, αλλά η απόδοσή του εξαρτάται από την ισορροπία του. Η επιλογή κομβοπροσανατολισμένης υλοποίησης μειώνει τον αριθμό κόμβων, ενώ η διαγραφή με inorder successor διατηρεί την ιδιότητα BST και στις πιο σύνθετες περιπτώσεις.

3 Άσκηση 3: Διαπεράσεις Δυαδικού Δέντρου

3.1 Εκφώνηση

(i) Υλοποιήστε συνάρτηση `levelOrder` για τη διέλευση δυαδικού δέντρου κατά επίπεδα χρησιμοποιώντας ουρά.

(ii) Υλοποιήστε επίσης τις διαπεράσεις προδιάταξης (preorder) και μεταδιάταξης (postorder). Να γίνει έλεγχος με κύριο πρόγραμμα.

3.2 Ανάλυση

Η διαπέραση ενός δυαδικού δέντρου είναι η διαδικασία επίσκεψης όλων των κόμβων με συγκεκριμένη σειρά. Υπάρχουν δύο βασικές κατηγορίες:

- **DFS (Depth-First Search):** Αναδρομικές διαπεράσεις που ακολουθούν ένα μονοπάτι μέχρι το φύλλο πριν επιστρέψουν. Περιλαμβάνει preorder, inorder, postorder.
- **BFS (Breadth-First Search):** Μη αναδρομική διαπέραση κατά επίπεδα (level order) που χρησιμοποιεί **ουρά**.

3.3 Διαπέραση κατά Επίπεδα (Level Order / BFS)

Η διαπέραση αυτή επισκέπτεται τους κόμβους επίπεδο-επίπεδο, από αριστερά προς τα δεξιά. **Δεν είναι αναδρομική** — χρησιμοποιεί ουρά (FIFO) για να εξασφαλίσει τη σωστή σειρά επεξεργασίας.

3.3.1 Αλγόριθμος

1. Εισάγουμε τη ρίζα στην ουρά
2. Όσο η ουρά δεν είναι άδεια:
   - Αφαιρούμε τον πρώτο κόμβο (dequeue)
   - Εκτυπώνουμε την τιμή του
   - Αν έχει αριστερό παιδί → το εισάγουμε στην ουρά (enqueue)
   - Αν έχει δεξί παιδί → το εισάγουμε στην ουρά (enqueue)

3.3.2 Υλοποίηση

```c
void levelOrder(struct Node* n) {
        if (n == NULL) return;

        struct Queue q;
        initializeQueue(&q);
        add_to_queue(&q, n);

        while (!isEmpty(&q)) {
                struct Node* current = remove_from_queue(&q);
                printf("%d ", current->data);

```

```
                if (current->left != NULL)
                add_to_queue(&q, current->left);
                if (current->right != NULL)
                add_to_queue(&q, current->right);
        }
}
```

3.4 Προδιάταξη (Preorder)

Σειρά επίσκεψης: **Ρίζα → Αριστερό → Δεξί**
Χρήσιμη για: αντιγραφή δέντρου, παραγωγή prefix έκφρασης.

```
void preorder(struct Node* n) {
        if (n == NULL) return;
        printf("%d ", n->data);   /* 1. Ριζα */
        preorder(n->left);        /* 2. Αριστερα */
        preorder(n->right);       /* 3. Δεξια */
}
```

3.5 Μεταδιάταξη (Postorder)

Σειρά επίσκεψης: **Αριστερό → Δεξί → Ρίζα**
Χρήσιμη για: διαγραφή δέντρου, παραγωγή postfix έκφρασης.

```
void postorder(struct Node* n) {
        if (n == NULL) return;
        postorder(n->left);       /* 1. Αριστερα */
        postorder(n->right);      /* 2. Δεξια */
        printf("%d ", n->data);   /* 3. Ριζα */
}
```

3.6 Υλοποίηση Ουράς

Η ουρά υλοποιείται με **πίνακα σταθερού μεγέθους** (γραμμική ουρά).

```
void add_to_queue(struct Queue *q, struct Node* n) {
        if (isFull(q)) { printf("Queue is full\n"); return; }
        q->rear++;
        q->items[q->rear] = n;
}

struct Node* remove_from_queue(struct Queue *q) {
        if (isEmpty(q)) { printf("Queue is empty\n"); return NULL; }
        q->front++;
        return q->items[q->front];
}
```

3.6.1 Γραμμική vs Κυκλική Ουρά

Η παρούσα υλοποίηση είναι **γραμμική**: μόλις ο `rear` φτάσει στο τέλος του πίνακα, η ουρά θεωρείται γεμάτη ακόμα κι αν υπάρχουν κενές θέσεις στην αρχή.

Στο level order traversal αυτό δεν αποτελεί πρόβλημα, καθώς κάθε κόμβος εισάγεται **ακριβώς μία φορά**. Για γενικότερη χρήση θα προτιμούσαμε **κυκλική ουρά** με:

```
        q->rear = (q->rear + 1) % BUFFER;
```

3.7 Αποτελέσματα Εκτέλεσης

Το δέντρο κατασκευάζεται από το αρχείο `numbers3.txt`:

50, 25, 75, 10, 30, 60, 90, 5, 15, 27, 35, 55, 65, 85, 95, 1, 8, 12, 18, 28

```
Level Order:
50 25 75 10 30 60 90 5 15 27 35 55 65 85 95 1 8 12 18 28
Preorder:
50 25 10 5 1 8 15 12 18 30 27 28 35 75 60 55 65 90 85 95
Postorder:
1 8 5 12 18 15 10 28 27 35 30 25 55 65 60 85 95 90 75 50
```

Σχήμα 4: Αποτελέσματα διαπεράσεων του δέντρου

3.8 Συμπεράσματα

Οι διαπεράσεις αποτελούν θεμελιώδη εργαλεία επεξεργασίας δέντρων. Η επιλογή μεθόδου εξαρτάται από το πρόβλημα: το level order είναι ιδανικό για επεξεργασία επιπέδων, το preorder για αντιγραφή/σειριοποίηση, και το postorder για διαγραφή ή υπολογισμό postfix εκφράσεων.

4 Άσκηση 4: Ύψος και Βάθος Δυαδικού Δέντρου

4.1 Εκφώνηση

Αξιοποιώντας τις διαπεράσεις του προηγούμενου ερωτήματος, υλοποιήστε:

- Συνάρτηση υπολογισμού **ύψους** δυαδικού δέντρου
- Συνάρτηση υπολογισμού και εκτύπωσης **βάθους** κάθε κόμβου

4.2 Ανάλυση

4.2.1 Ύψος Δέντρου

Το **ύψος** ενός δυαδικού δέντρου ορίζεται ως ο αριθμός **ακμών** στο μακρύτερο μονοπάτι από τη ρίζα ως ένα φύλλο.
Σύμβαση:

- Κενό δέντρο (`NULL`): ύψος $= -1$
- Φύλλο (κόμβος χωρίς παιδιά): ύψος $= 0$

4.2.2 Βάθος Κόμβου

Το **βάθος** ενός κόμβου είναι ο αριθμός **ακμών** από τη ρίζα ως τον κόμβο.
Σύμβαση:

- Ρίζα: βάθος $= 0$
- Παιδιά ρίζας: βάθος $= 1$
- Εγγόνια ρίζας: βάθος $= 2$, κ.ο.κ.

4.2.3 Σχέση Ύψους και Βάθους

Για οποιονδήποτε κόμβο v σε **πλήρως ισορροπημένο** δέντρο ισχύει:

$$\text{ύψος}(v) + \text{βάθος}(v) = \text{ύψος δέντρου}$$

4.3 Υλοποίηση

4.3.1 Ύψος Δέντρου

Η συνάρτηση υπολογίζει αναδρομικά το ύψος από τα φύλλα προς τη ρίζα (bottom-up), επιστρέφοντας σε κάθε βήμα $1 + \max(\text{ύψος_αριστερά}, \text{ύψος_δεξιά})$:

```
int treeHeight(struct Node* n) {
        if (n == NULL) return -1;

        int left  = treeHeight(n->left);
        int right = treeHeight(n->right);

        return 1 + (left > right ? left : right);
}
```

Παράδειγμα υπολογισμού για τα δεδομένα μας:

Κόμβος	**Ύψος**
1 (φύλλο)	0
5	1
10	2
25	3
50 (ρίζα)	4

4.3.2 Βάθος Κόμβων

Χρησιμοποιεί preorder διάσχιση (top-down): ξεκινά από τη ρίζα με βάθος 0 και αυξάνει κατά 1 σε κάθε επίπεδο:

```
void treeDepth(struct Node* n, int depth) {
        if (n == NULL) return;

        printf("Node: %2d,  Depth: %d\n", n->data, depth);

        treeDepth(n->left,  depth + 1);
        treeDepth(n->right, depth + 1);
}
```

4.4 Δεδομένα Εισόδου

Το δέντρο κατασκευάζεται από το αρχείο `numbers4.txt`:

50, 25, 75, 10, 30, 60, 90, 5, 15, 27, 35, 55, 65, 85, 95, 1, 8, 12, 18, 28

Το δέντρο είναι **πλήρως ισορροπημένο** (perfect binary tree στα επίπεδα 0--3, με επιπλέον κόμβους στο επίπεδο 4), οπότε το αναμενόμενο ύψος είναι **4**. Το μακρύτερο μονοπάτι είναι π.χ.:

$$50 \rightarrow 25 \rightarrow 10 \rightarrow 5 \rightarrow 1$$

4.5 Αποτελέσματα Εκτέλεσης

```
Tree height = 4

Node: 50,  Depth: 0
Node: 25,  Depth: 1
Node: 10,  Depth: 2
Node:  5,  Depth: 3
Node:  1,  Depth: 4
Node:  8,  Depth: 4
Node: 15,  Depth: 3
Node: 12,  Depth: 4
Node: 18,  Depth: 4
Node: 30,  Depth: 2
Node: 27,  Depth: 3
Node: 28,  Depth: 4
Node: 35,  Depth: 3
Node: 75,  Depth: 1
Node: 60,  Depth: 2
Node: 55,  Depth: 3
Node: 65,  Depth: 3
Node: 90,  Depth: 2
Node: 85,  Depth: 3
Node: 95,  Depth: 3
```

Σχήμα 5: Ύψος δέντρου και βάθος κόμβων

Αναμενόμενη έξοδος (μερική):

```
Tree height = 4

Node: 50,  Depth: 0
Node: 25,  Depth: 1
Node: 10,  Depth: 2
Node:  5,  Depth: 3
Node:  1,  Depth: 4
Node:  8,  Depth: 4
Node: 15,  Depth: 3
...
```

4.6 Συμπεράσματα

Ο υπολογισμός ύψους και βάθους αξιοποιεί τις αναδρομικές τεχνικές διαπέρασης της Άσκησης 3. Το ύψος υπολογίζεται **bottom-up** (από φύλλα προς ρίζα), ενώ το βάθος **top-down** (από ρίζα προς φύλλα). Οι συναρτήσεις αυτές αποτελούν τη βάση για πιο σύνθετες λειτουργίες όπως έλεγχος ισορροπίας δέντρου και AVL rotations.

5 Άσκηση 5: Ανάπλαση Δυαδικού Δέντρου από Διαπεράσεις

5.1 Εκφώνηση

Δίνεται δυαδικό δέντρο με μοναδικούς χαρακτήρες. Να εξηγηθεί και να υλοποιηθεί η ανάπλαση του δέντρου όταν δίνονται:

- Ενδοδιαπέραση (Inorder) + Μεταδιαπέραση (Postorder)
- Ενδοδιαπέραση (Inorder) + Προδιαπέραση (Preorder)

Να επιβεβαιωθεί το αποτέλεσμα καλώντας μία από τις συναρτήσεις διαπέρασης της Άσκησης 3.

5.2 Θεωρητική Ανάλυση

Η ανακατασκευή δυαδικού δέντρου είναι δυνατή μόνο με **συγκεκριμένους συνδυασμούς** διαπεράσεων, υπό την προϋπόθεση ότι οι χαρακτήρες είναι **μοναδικοί**.

Συνδυασμός	**Μοναδική ανακατασκευή;**
Inorder + Preorder	✓Ναι
Inorder + Postorder	✓Ναι
Preorder + Postorder (μόνο)	× Όχι πάντα

Το **inorder είναι απαραίτητο**: είναι η μόνη διαπέραση που χωρίζει καθαρά το αριστερό από το δεξί υποδέντρο. Χωρίς αυτό δεν μπορούμε να προσδιορίσουμε μοναδικά τη δομή.

5.2.1 Λογική Ανακατασκευής

Και στις δύο περιπτώσεις η ιδέα είναι ίδια:

1. **Βρίσκουμε τη ρίζα** από το preorder (πρώτο στοιχείο) ή postorder (τελευταίο στοιχείο)
2. **Εντοπίζουμε τη ρίζα στο inorder** — ό,τι είναι αριστερά της ανήκει στο αριστερό υποδέντρο, ό,τι δεξιά στο δεξί
3. **Εφαρμόζουμε αναδρομικά** για κάθε υποδέντρο

5.2.2 Παράδειγμα Χειροκίνητης Ανακατασκευής

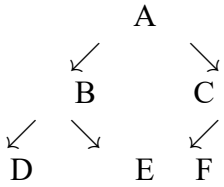
Δεδομένα:

- Inorder: `D B E A F C`
- Preorder: `A B D E C F`
- Postorder: `D E B F C A`

Από το preorder: ρίζα = `A`. Στο inorder η `A` βρίσκεται στη θέση 3:

$$\underbrace{D\ B\ E}_{\text{αριστερό}} \mid A \mid \underbrace{F\ C}_{\text{δεξί}}$$

Εφαρμόζουμε αναδρομικά και παίρνουμε:



5.3 Αλγόριθμος Ανακατασκευής από Preorder + Inorder

1. Το `preorder[preIndex]` είναι η ρίζα του τρέχοντος υποδέντρου
2. Εντοπίζουμε τη ρίζα στο `inorder[left..right]`
3. Αναδρομή **πρώτα αριστερά**, μετά δεξιά (ακολουθεί τη σειρά του preorder)

```
struct Node* buildTreePreRecursive(char inorder[], char preorder[],
int *preIndex, int left, int right) {
        if (left > right) return NULL;

        char rootVal = preorder[*preIndex];
        (*preIndex)++;
        struct Node* root = createNode(rootVal);

        int index = search(inorder, rootVal, left, right);

        root->left  = buildTreePreRecursive(inorder, preorder,
        preIndex, left, index - 1);
        root->right = buildTreePreRecursive(inorder, preorder,
        preIndex, index + 1, right);
        return root;
}
```

5.4 Αλγόριθμος Ανακατασκευής από Postorder + Inorder

1. Το `postorder[postIndex]` είναι η ρίζα (διαβάζουμε **ανάποδα**)
2. Εντοπίζουμε τη ρίζα στο `inorder[left..right]`
3. Αναδρομή **πρώτα δεξιά**, μετά αριστερά — κρίσιμο! (λόγω ανάποδης ανάγνωσης του postorder)

```
struct Node* buildTreePostRecursive(char inorder[], char postorder[],
int *postIndex, int left, int right) {
        if (left > right) return NULL;

        char rootVal = postorder[*postIndex];
        (*postIndex)--;
        struct Node* root = createNode(rootVal);

        int index = search(inorder, rootVal, left, right);

```

```
        root->right = buildTreePostRecursive(inorder, postorder,
        postIndex, index + 1, right);
        root->left  = buildTreePostRecursive(inorder, postorder,
        postIndex, left, index - 1);
        return root;
}
```

5.5 Επιβεβαίωση Ορθότητας

Για επιβεβαίωση εφαρμόζουμε και τις τρεις διαπεράσεις (από Άσκηση 4) στο ανακατασκευασμένο δέντρο και συγκρίνουμε με τα αρχικά δεδομένα:

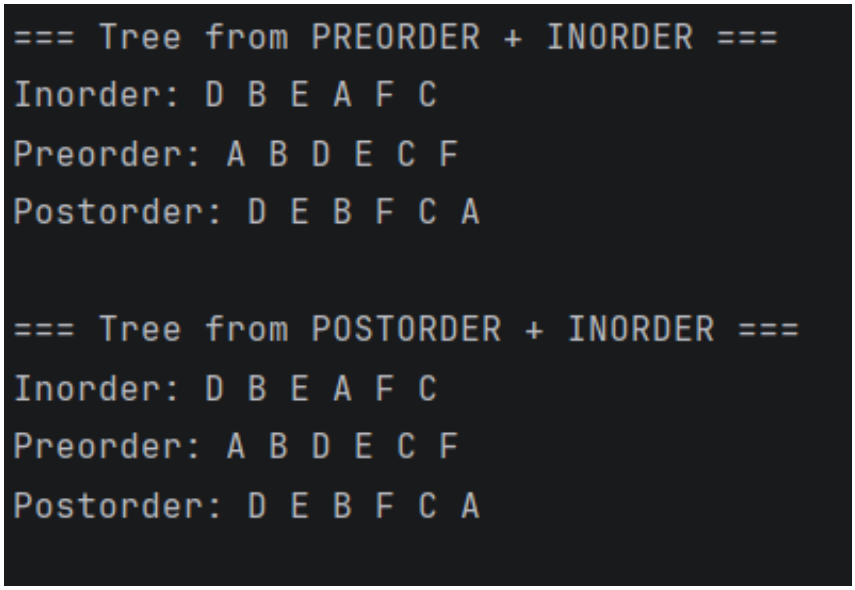
```
printf("Inorder: ");
inorderPrint(preRoot);

printf("Preorder: ");
preorderPrint(preRoot);

printf("Postorder: ");
postorderPrint(preRoot);
```

5.6 Δεδομένα Ελέγχου και Αποτελέσματα

Διαπέραση	**Τιμή**
Inorder	D B E A F C
Preorder	A B D E C F
Postorder	D E B F C A



Σχήμα 6: Επιβεβαίωση ανακατασκευασμένου δέντρου

Και τα δύο δέντρα (από preorder και postorder) παράγουν το ίδιο inorder `D B E A F C`, επιβεβαιώνοντας την ορθότητα της ανακατασκευής.

5.7 Συμπεράσματα

Η ανακατασκευή δυαδικού δέντρου από δύο διαπεράσεις είναι εφικτή μόνο όταν η μία είναι το **inorder**, καθώς αυτό παρέχει τη μοναδική πληροφορία διαχωρισμού αριστερού/δεξιού υποδέντρου. Η τεχνική αυτή χρησιμοποιείται σε parsing, compilers και προβλήματα σειριοποίησης δέντρων.